JANUAR

I - 2 Metode pristupanja podacima memorije (sekvencijalno, direktno, nasumicno, asocijativno)

➔ Sekvencijalno pristupanje
- Pristupanje mora da se obavlja odredjenim linearnim redom
- Memorija je organizovana u jedinice podataka tzv. zapise
- Informacije o adresama se koriste za razdvajanje zapisa
- Zajednicki mehanizam za citanje i upisivanje, pri cemu se vrsi pomeranje sa trenutnog nazeljeni polozaj preskakanjem svih zapisa izmedju
- Vreme pristupa je promenljivo
- Magnetne trake koriste sekvencijalno pristupanje

➔ Direktno pristupanje
- Zajednicki mehanizam za citanje i upisivanje
- Pojedinacni blokovi ili zapisi imaju jedinstvenu adresu na osnovu fizicke lokacije
- Pristupanje se obavlja direktnim pristupom blizu trazenih podataka, a zatim sepretrazuje kako bi se stiglo do konacne lokacije
- Vreme pristupa je promenljivo
- Koriste ga magnetni diskovi

➔ Nasumicno pristupanje
- Svaka lokacija u memoriji koja moze da se adresira ima zaseban fizicki ozicenimehanizam za adresiranje
- Vreme pristupa za odredjenu lokaciju zavisi od prethodnih pristupa i nepromenljivo je
- Bilo koja lokacija moze da se direktno adresira i da joj se direktno pristupi
- Koriste ga glavna memorija i neki sistemi kes memorija

➔ Asocijativno pristupanje
- Tip memorije sa nasumicnim pristupom, poredjenjem zeljenih lokacija bita unutar recipronalazi se podudarnost (ovo se radi za sve reci istovremeno)
- Rec se preuzima na osnovu dela njenog sadrzaja, a ne na osnovu njene adrese
- Vreme pristupa odredjenoj lokaciji ne zavisi od lokacije i prethodnih pristupa tj.nepromenljivo je
- Koriste ga kes memorije

II – 4 Tehnike preslikavanja iz glavne u keš memoriju:

➔ Direktno
- Svaki blok glavne memorije preslikava u tačno jednu mogudu liniju keš memorije kojamu je pridružena
- To preslikavanje se

  izražava sa:i = j

  mod m
  - i je broj linije keš memorije
  - j je broj bloka glavne memorije

- m je broj linija u keš memoriji

- Svaki blok glavne memorije preslikava se u jednu zasebnu liniju keš memorije. Sledećih m blokova glavne memorije preslikava se u keš memoriju na isti način tj. blok Bm u linijuL0, blok Bm+1 u liniju L1 itd.
- Ova tehnika je jednostavna i nije skupa za primenu
- Mana: može se desiti da program više puta pristupa rečima iz dva različita bloka koji sepreslikavaju u istu liniju, tako da ti blokovi stalno menjaju mesto u keš memoriji i stopapogodaka je niska -> poznato kao treperenje (eng. trashing)

➔ Asocijativno

- Omogudava da se neki blok glavne memorije učita u bilo koju liniju keš memorije
- Da bi se utvrdilo da li se neki blok nalazi u keš memoriji , moraju se istovremenopretražiti sve linije keš memorije da bi se našlo podudaranje
  - Za tu namenu koristi se CAM memorija (content-addressable memory) tj. memorijakoja se adresira po sadržaju
- Postoji sloboda izbora koji blok se zamenjuje kad se novi blok učita u keš memoriju
- Nedostatak: složena logička kola neophodna za paralelno proveravanje svih oznaka linijakeš memorije

➔ Skupno asocijativno preslikavanje

- Koristi prednosti tehnike direktnog i asocijativnog preslikavanja dok umanjuje njihovenedostatke
- Keš memorija se posmatra kao da je organizovana iz više skupova pri čemu se svaki skupsastoji od više linija
- k-struko skupno asocijativno preslikavanje je definisano sa:

  $m = v*k$

  $i = j \bmod v$

  - i je broj skupa keš memorije
  - j je broj bloka glavne memorije
  - m je broj linija u keš memoriji
  - v je broj skupova
  - k je broj linija u svakom skupu
- Može se realizovati na dva načina:
- 1. pomocu v keš memorija sa asocijativnim preslikavanjem (v CAM memorija)
  - Blok glavne memorije B0 se preslikava u skup 0, pri čemu se neka reč iz ovog blokamože preslikati u bilo koju liniju u skupu 0 • blok Bj može da se preslika u bilo kojuod linija iz skupa j
- 2. pomocu k keš memorija sa direktnim preslikavanjem
  - Za svaku keš memoriju sa direktnim preslikavanjem kaže se da je struk koji sačinjavav linija
  - Prvih v linija glavne memorije direktno se preslikava u v linija nekog struka

IV – 5 Dinamičko particionisanje memorije – objašnjenje i primeri; načini vođenja evidencije

- U ovom slučaju veličine particija nisu fiksne, već kada se neki proces unosi u memorijudodeljuje mu se tačno onoliko memorije koliko mu treba
- Sistemi koji podržavaju dinamičke particije moraju voditi računa o slobodnoj i zauzetojmemoriji
- Kada novi proces dođe na red za izvršavanje, pronalazi se dovoljno velika slobodnaparticija, a zatim se iz nje izdvaja onoliko memorije koliko je procesu potrebno
- Nakon završetka izvršavanja, proces oslobađa particiju koju je koristio
- Šupljina je termin koji označava slobodni, kontinualni deo memorije
- Kada naiđe proces, sistem traži šuljinu dovoljno veliku da smesti čitav proces
- OS vodi evidenciju o:
  - Particijama koje su zauzete procesima
  - Slobodnim šupljinama u memoriji
- Načini vođenja evidencije:
  - Bit mape
  - Povezane liste
- Bit mape podrazumevaju da se memorija podeli na jednake delove i da se svakom od tihdelova pridruži po jedan bit
  - Pridruženi bit ima vrednost 1 ukoliko je odgovarajući deo memorije zauzet, u suprotnom je 0
  - Za smeštanje procesa u memoriju potrebno je pronaći dovoljno veliki niz nula u bitmapi
  - Prednost: implementacija bit mapa je jednostavna
  - Mana: vreme potrebno za pretraživanje i pronalaženje slobodne particije dosta veliko
- Evidencija u povezanoj listi se sastoji od čvorova, pri čemu svaki čvor sadrži sledeće:
    - Oznaka tipa memorije - da li je u delu memorije, na koji se čvor odnosi,smešten proces (P) ili je u pitanju slobodna memorija (H)
    - Početna adresa odgovarajućeg dela memorije
    - Dužina tog dela memorije
    - Pokazivač na sledeći čvor koji čuva informacije o narednom delu memorije
  - U nekim slučajevima se liste slobodnih i zauzetih delova memorije implementirajuodvojeno
- Time je olakšana pretraga slobodnih delova, ali je mana ažuriranje dve liste

V – 7 Markirajući sakupljač otpadaka (mark sweep) – kako funkcioniše, objašnjenje obe faze (faze markiranja i faze brisanja) i karakteristike

- Spada u grupu sakupljača otpadaka čije pronalaženje otpadaka se bazira na praćenjudostupnosti objekata na heap-u (eng. tracing garbage collector)
- Pretraga kreće počev od korenih pokazivača, pa se dalje prati niz referenci do objekata
- Objekat je dostupan ako je referenciran od strane korenih pokazivača ili referenciran odstrane drugog objekta koji je dostupan
- oni objekti na koje ne postoje reference iz skupa korenih pokazivača, bilo direktne iliindirektne reference, su otpaci čija memorija se može osloboditi
- najstariji tip sakupljača otpadaka razvijen 1960. od strane John McCarthy u okviruprogramskog jezika LISP
- Sakupljanje otpadaka ima 2 faze:

➔ 1. faza: mark (faza obeležavanja) - obuhvata pronalaženje i markiranje aktivnih ćelija

- Pod markiranjem se podrazumeva upisivanje jedinice u određeni bit (bit za markiranje) koji je pridružen svakoj ćeliji. Taj bit se nalazi ili u samoj ćeliji ili u bit-mapi
- postupak markiranja u stvari predstavlja kretanje po dubini kroz grafovskustrukturu, počevši od skupa korenih pokazivača
- Na kraju ovog procesa su markirane sve ćelije koje su naslednice korena

- 1. mark faza - Postupak markiranja:
  - kreće se od nekog korenog pokazivača i markira se ćelija na koju on pokazuje (markbit = 1)
  - Ako markirana ćelija sadrži elementarni podatak, proces se završava
  - Ako markirana ćelija sadrži pokazivače, proces markiranja se primenjuje rekurzivnona ćelije na koje pokazuju ti pokazivači
- postupak se ponavlja za svaki postojeći koreni pokazivač
- na ovaj način sve aktivne ćelije u heap-u biće markirane

➔ 2. faza – sweep faza (faza brisanja)

- Heap se sekvencijalno pretražuje pri čemu se ćelije koje nisu markirane, a to suslobodne ćelije i otpaci, povezuju u listu slobodnih ćelija -> oslobađa se njihovamemorija
- u ovoj fazi se bit za markiranje u svim aktivnim ćelijama postavlja na 0 kako bi sepripremili za naredni ciklus algoritma

➔ Karakteristike mark-sweep algoritma

- Lako se implementira
- U stanju je da reciklira i ciklične strukture, što je naročito bitno kod implementacijeredukcije grafa
- Nedostaci:
- kada se aktivira proces sakupljanja otpadaka mora da se zaustavi program (Mutator) kojise izvršava

    - Ovo može izazvati da se program periodično „zamrzne“ što je problematično uslučaju real-time i aplikacija gde je vreme izvršavanja kritično
- Proces markiranja je rekurzivan -> sporo se izvršava
- Sve aktivne ćelije se posećuju dva puta (i u prvoj i u drugoj fazi), a neaktivne jednom (udrugoj fazi)
    - -> brzina sakupljanja otpadaka ne zavisi samo od broja aktivnih ćelija već i odcelokupne veličine heap-a
- Moguća fragmentacija memorije jer se ne radi kompaktovanje
- Često se koristi kao osnova za druge algoritme

VI – 1 TCP/UDP funkcionalnost i TCP/UDP adresiranje. Pojam soketa (utičnice).

➔ TCP/UDP funkcionalnost
- TCP: Connection-oriented, stream, reliable, error-free delivery
    - TCP paketi su označeni brojem sekvence, koji se ponavlja ali u dužem periodu
    - Isporuka se organizuje po tom broju
    - U slučaju greške, TCP radi retransmisiju paketa
- UDP: connectionless, message, unreliable datagram service
    - Podrazumeva kontrolu isporuke na aplikacionom nivou
    - Jednostavniji i efikasniji
    - Manja kašnjenja pogoduju multimedia aplikacijama
    - Npr, izgubljeni VoIP paket se ne zanavlja
- Oba protokola su FDX, klijent-server usluga

➔ TCP/UDP adresiranje
- Socket = IP adresa + broj porta (prolaza)
    - Port je broj n[1 - 65535] dodeljen korisničkoj aplikaciji
    - Prvih 1023 “portova” su rezervisani
- Poruke se pre slanja dele na pakete
    - Dodaje im se TCP/UDP zaglavlje
- Na obe strane se formiraju Tx/Rx baferi
    - Kontrola toka sprečava prepisivanje/prepunjavanje na prijemu

➔ Soket (socket, uticnica)
- Preko soketa procesi šalju poruke u mrežu
- Standardna API sprega (Application Programming Interface)
- Multipleksiranje / demultipleksiranje poruka

VII – 6 Heterogeno računarstvo – pojam i tipična konfiguracija

- Heterogeno računarstvo koristi više od jednog tipa procesora (npr. CPU i GPU). Naziva sei hibridno računarstvo ili “ubrzano” (accelerated computing). Razlozi : energetska
  efikasnost, veća propusnost ... Server se obično zove host, a GPU device Veza preko PCIeje diskretna a postoji i integrisana verzija CPU i GPU na jednom čipu
- Tipična serverska konfiguracija ima jedan ili više CPU-a (obično dva) koji komuniciraju sajednim ili više grafičkih procesora preko PCIe busa.
- CPU: velike keš memorije, nekoliko niti po jezgru, umerena širina opsega memorijezasnovana uglavnom na keš memoriji i prefetching-u
- GPU: male keš memorije, veliki broj niti, velika širina opsega memorije zasnovana namulti-threading karakteristikama

VII – 15 Tipičan tok CUDA programa

1. Load data into CPU memory
   fread/rand
2. Copy data from CPU to GPU memory
   cudaMemcpy(..., cudaMemcpyHostToDevice)
3. Call GPU kernel
   yourkernel<<>>(...)
4. Copy results from GPU to CPU memory.
   cudaMemcpy(...,
   cudaMemcpyDeviceToHost)
5. Use results on CPU

FERBRUAR

I – 4 Princip lokalnsti: vremenska i prostorna lokalnost

Princip lokalnosti ili lokalnost pozivanja memorije – memorija koju procesor poziva tokomizvršavanja programa, kako za instrukcije tako i za podatke, teži tome da se grupiše

- primeri: programske petlje, potprogrami koji se ponavljaju; operacije sa nizovima,matricama

Dva oblika lokalnosti:

➔ Vremenska lokalnost – sklonost programa da u bliskoj bududnosti poziva one jedinice memorije koje je pozivao u neposrednoj prošlosti

➔ Prostorna lokalnost – sklonost programa da pristupa jedinicama memorije čije su adrese jedna blizu druge tj. odražava sklonost procesora da instrukcijama pristupa

redom kao i dalokacijama podataka pristupa redom (npr. obrada tabele podataka)

Oba oblika lokalnosti odnose se i na instrukcije i podatke u memoriji: • Vremenska lokalnost podataka i vremenska lokalnost instrukcija • Prostorna lokalnost podataka i prostorna lokalnostinstrukcija

IV – 8 Adrese u memoriji – fizičke i logičke adrese; jedinica za upravljanje memorijom (MMU); registarrelokacije

- Prilikom zamene mesta neki proces se verovatno neće učitati na isto mesto kad se vratinazad u memoriju (kao Proces 2 u primeru za dinamičko particionisanje)
- Dodatno, ako se koristi sabijanje neki proces se može pomeriti dok je u glavnoj memoriji
- Proces u memoriji se sastoji od instrukcija i podataka
- kod sistema sa višeprogramskim radom, veći broj procesa deli operativnu memorijuračunara
    - programer ne može unapred odrediti fiksne memorijske lokacije za smeštajprograma
    - adrese podataka i instrukcija su promenljive jer se menjaju svaki put kad se procespremešta
      -> rešenje je uvođenje razlike između logičkih adresa i fizičkih adresa

➔ Logička adresa: izražava se kao relativni položaj u odnosu na početak programa

- Instrukcije u programu sadrže samo logičke adrese
- Logičke adrese su generisane od strane procesora dok se program izvršava
    - Naziva se još i virtualna adresa jer ona fizički ne postoji
- logički (virtualni) adresni prostor je skup svih logičkih adresa koje generiše jedanprogram

➔ Fizička adresa: stvarna adresa u memoriji

- Identifikuje fizičku lokaciju u memoriji
- Procesor pri izvršavanju procesa automatski pretvara logičke adrese u fizičke adrese tako što na svaku logičku adresu dodaje trenutni početni položaj procesa koji se nazivaosnovna adresa (base address)

➔ Jedinica za upravljanje memorijom (Memory Management Unit, MMU) preslikava logičkeadrese u fizičke adrese prilikom izvršavanja programa

- MMU je hardverski uređaj koji mapira virtuelni adresni prostor u fizički adresniprostor

- Jednostavni MMU se može predstaviti kao uređaj koji poseduje poseban registar koji senaziva registar relokacije
    - Korisnički program barata logičkim adresama, koje on vidi u opsegu od [0, max]

    - MMU dodaje vrednost registra relokacije (R) na logičku adresu i tako generiše fizičkuadresu
      • Vrednost registra relokacije R je trenutna fizička adresa početka

programa umemoriji
    - MMU preslikava logički adresni prostor koji se nalazi u opsegu [0, max] u fizičkiadresni prostor u opsegu [R+0, R+max]
- MMU i registar relokacije
- R označava vrednost registra relokacije tj. fizičku adresu početka programa u memoriji

Fizička adresa = logička adresa + R

IV – 15 Segmenatcija memorije – osnovni principi, logička adresa; tabela segmenata; mapiranje logičkeu fizičku adresu

- straničenje memorije je nevidljivo za programera i služi da omogući veći adresni prostor
- Segmentacija je obično vidljiva za programere i nudi se kao pogodnost za organizovanjeprograma i podataka, kao i dodeljivanje privilegija i atributa zaštite instrukcijama i podacima
- Segmentcija omogućava programerima da memoriju vide kao da se sastoji od višeadresnih prostora ili segmenata • Segmenti su promenljive tj. dinamičke veličine
- Svakom segmentu mogu da se dodele prava za pristupanje i korišćenje
- Adrese za pristup memoriji su u obliku:
    - (Broj segmenta, pomeraj)
- korisnički program se može posmatrati kao skup različitih logičkih celina – segmenata
    - Na primer, neki od njih su: glavni program, stek, hip, biblioteke, globalni podaci itd.
- Osnovna ideja segmentacije je da se svakom segmentu dodeli poseban memorijskiprostor
- Segmenti se mogu naći bilo gde u memoriji, ali jedan segment mora biti u neprekidnommemorijskom bloku
- Nije neophodno da segmenti budu iste veličine, već se svakom segmentu dodeljujetačno onoliko memorije koliko je potrebno
- logički adresni prostor se sastoji od skupa segmenata
- svaki segment ima jedinstveno ime (ID) i dužinu
- logička adresa se se sastoji od dva dela:
    - Broj segmenta (broj koji predstavlja ID segmenta)
    - pomeraja unutar segmenta
- kod segmenatacije je prisutna eksterna fragmentacija
- slobodna memorija se ne može iskoristiti za smeštaj segmenta ukoliko ne postojidovoljno velika šupljina
- problem se može umanjiti sažimanjem memorije
- kod metode segmentacije logičke adrese su dvodimezionalne
- mapiranje u fizičke adrese se obavlja preko tabele segmenata
- svaki ulaz u tabeli segmenata sadrži dva parametra:
    - baznu adresu segmenta (početna fizičku adresu segmenta u memoriji)
    - ograničenje (limit) segmenta (tj. dužina segmenta)

- mapiranje logičke adrese u fizičku se vrši na sledeći način:
  - broj segmenta (s) se koristi kao ulaz u tabelu segmenta
  - iz tabele se čitaju bazna adresa segmenta (b) i limit (l)
  - ako je pomeraj (d) veći od limita (d > l) -> greška u adresiranju
  - ako je pomeraj manji od limita (d < l): -> fizička adresa = bazna adresa + pomeraj

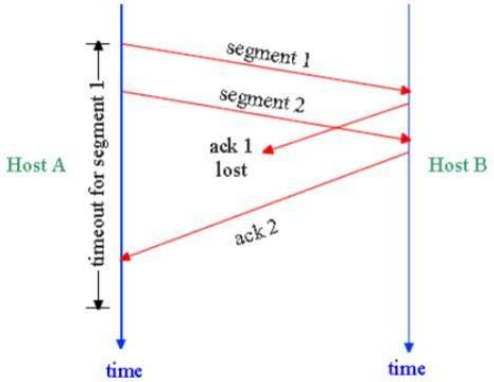
V – 11 Kopirajući tip sakupljača otpadaka – objašnjenje na primeru dvoprostornog sakupljača otpadaka.Prednosti i mane.

- 3.1 Dvoprostorni sakupljač otpadaka
- Heap je podeljen na 2 dela:
  - 1. Izvorni deo- služi za smeštanje podataka nastalih u toku rada programa
  - 2. Ciljni deo– koristi se samo u toku rada sakupljača otpadaka – u njega se prepisujesadržaj aktivnih objekata iz izvornog dela
- Uslov: Veličina ciljnog dela mora biti ista kao veličina izvornog dela
- Na kraju procesa sakupljanja, izvorni i ciljni deo heap-a zameniće uloge, tj. ciljni deopostaće izvorni i obrnuto
- Posao se obavlja u dve faze:
- 1. Pronalaženje aktivnih ćelija u izvornom delu heap-a, po metodi mark sweepalogoritma (rekurzivno markiranje)
- Pretraga aktivnih ćelija kreće od korenih pokazivača
- Svaka aktivna ćelija se kopira u prvu slobodnu ćeliju ciljnog dela heap-a
- premeštene ćelije formiraju kontinualan blok, čime se eliminiše pojava fragmentacije ->ovaj postupak naziva se i sažimanje, kompakcija
- 2. Ažuriranje pokazivača
- Nakon premeštanja ćelije, u njenu staru lokaciju upisuje se adresa nove lokacije
  - ta adresa se naziva adresa prosleđivanja (forwarding adress)
- Prednosti:
  - Svaka aktivna ćelija posećuje se dva puta (jednom u prvoj i jednom u drugoj fazi), alise neaktivne ćelije ne posećuju
  - -> brzina rada sakupljača otpada zavisi samo od broja aktivnih ćelija
  - -> brži od markirajućeg sakupljača
  - Nema fragmentacije memorije i lokalnost referenci je dobra
  - Jednostavan za implementaciju
- Mane:
  - Proces sakupljanja otpadaka je rekurzivan
  - Iskorišćenost heap-a je najviše 50%, (ciljni deo se ne koristi za smeštanje podataka utoku izvršavanja programa)

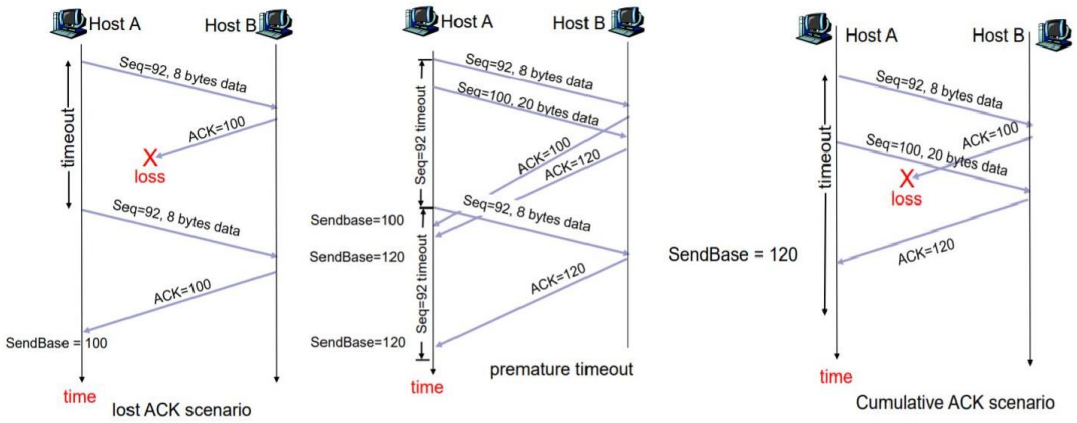
mutator mora da se zaustavi i sačeka da sakupljač otpadaka završi svoj posao

VI – 6 Potvrda paketa i scenariji zanavljanja kod TCP protokola

- Pri slanju segmenta se startuje timer i čeka potvrda
- Ako potvrda izostane vrši se retransmisija
- Koristi mehanizam klizajućeg prozora



➔ Scenariji zanavljanja



VII – 3 Poređenje karaketristika CPU i GPU procesora

- Centralna procesorska jedinica - CPU
  - Optimizovana za pristup keširanom skupu podataka sa malim kašnjenjem
  - Kontrolna logika za neispravno izvršavanje
  - Izvršenje desetak niti
- Grafička procesorska jedinica - GPU
  - Optimizovana za paralelan rad sa podacima i povećanje propusnosti
  - Arhitektura tolerantna na memorijska kašnjenja
  - Izvršavanje desetak hiljada niti
- Različiti ciljevi proizvode različite dizajne
  - CPU mora biti efikasan za svaki problem, bilo da je paralelan ili ne
  - GPU pretpostavlja da je radno opterećenje visoko paralelno
- CPU: minimizira kašnjenje dobijeno od jedne niti
  - Velike keš memorije na čipu

    - Sofisticirana kontrolna logika
- GPU: maksimizira propusnost svih niti
    - Broj niti koje se izvršavaju je ograničen resursima => potrebno je dosta resursa(registri, propusnost, itd.)
    - multithreading može da sakrije kašnjenja => preskoči veliki keš
    - Deli kontrolnu logiku između više niti
- CPU - task paralelizam
- GPU - data paralelizam
- Kompleksnost GPU-a je uveliko smanjena (nemaju inteligenciju CPU-a, nemaju prefetch,nemaju branch prediction …)
- GPU procesori namenjuju najviše svojih tranzistora za procesiranje, sledi da su oniidealni za računarski-intenzivna, visoko paralelizovana izračunavanja
    - Efekti u video igrama, računarske naučne simulacije, procesiranje slike i mašinskoučenje, linearna algebra ...

VII – 14 CUDA programiranje – host kod i kernel kod

- CUDA program je sastavljen iz dva dela:
    - 1. host kod koji se izvršava na CPU koji je povezan sa GPU
    - 2. Kernel kod koji se izvršava na GPU

CUDA programiranje – host kod

Na nivou host koda, postoje bibliotečke rutine za:

- alokacija memorije na grafičkoj kartici
- transfer podataka ka/od device memorije, uključujući
    - •konstantne
    - •obične podatke data
- error-checking
- timing

Postoji specijalna sintaksa za startovanje više instanci kernel procesa na GPU.

```
// Allocate pointers for host and device memory
float *h_input, *h_output;
float *d_input, *d_output;

// malloc() host memory (this is in your RAM)
h_input= (float*) malloc(mem_size);
h_output= (float*) malloc(mem_size);

// allocate device memory input and output arrays
cudaMalloc((void**)&d_input,  mem_size);
cudaMalloc((void**)&d_output, mem_size);

// Do something here!

// cleanup memory
free(h_input);
free(h_output);
cudaFree(d_input);
cudaFree(d_output);
```

```
// Copy host memory to device input array
cudaMemcpy(d_input, h_input, mem_size, cudaMemcpyHostToDevice);

// Do something on the GPU

// copy result from device to host
cudaMemcpy(h_output, d_output, mem_size,
cudaMemcpyDeviceToHost);
```

CUDA programiranje – host kod

U svojoj najprostijoj formi specijalna sintaksa izgleda kao:

kernel_routine<<<gridDim, blockDim>>> (args);

gridDim je broj instanci kernela (gridDim veličina rešetke)

blockDim je broj niti u svakoj instanci (blockDim veličina bloka)

Args je ograničen broj argumenata, obično uglavnom pokazuje na nizove u grafičkoj memoriji i na neke konstante koje se kopiraju po vrednosti

Opštija forma dozvoljava da gridDim i blockDim budu 2D ili 3D da bi pojednostavili program aplikacije

```
__global__ void helloworld_GPU(void){
  printf("Hello world!\n");
}

Int main(void) {

// run CUDA kernel
helloworld_GPU<<<1,1>>>();
?
return(0);
}
```

Kernel kod

```
#include <helper_cuda.h>

__global__ void my_first_kernel(float *x)
{
Int tid= threadIdx.x+ blockDim.x*blockIdx.x;
x[tid]  = (float) threadIdx.x;
}
```

- __global__ identifikator kaže to je kernel funkcija

- Svaka nit postavlja jedan element x niza.

- Unutar svakog bloka niti, threadIdx.x uzima vrednosti iz opsega od 0 do blockDim.x-1, tako da svaka nit ima jedinstvenu vrednost *tid*.

MART

I – 6 Pristupanje memorije sa dva i tri nivoa – princip i prosecno vreme pristupa

➔ Pristupanje memoriji sa dva nivoa

- Uzedemo kao najjednostavniji primer, koji se retko primenjuje u savremnim sistemima,računar koji ima samo jedan nivo keš memorije povezan sa glavnom memorijom
  - Keš memorija – kao posredna (eng.buffer) memorija između procesora i glavnememorije -> nastaju dva nivoa unutrašnje memorije
  - Mehanizam keširanja glavne memorije ostvaruje se u hardveru i obično je nevidljivza operativni sistem
- Nivo memorije M1 je manji, brži i skuplji u odnosu na nivo memorije M2
- Nivo M1 se koristi kao privremeno skladište za deo sadržaja vedeg nivoa M2
- Kada se pozove memorija, prvo se pokuša pristupanje određenom podatku na nivouM1. Ako uspe, to je brz pristup
- Ako ne uspe, tada se blok memorijskih lokacija kopira sa nivoa M2 na nivo M1 , a zatimse traženom podatku pristupa na nivou M1

- Zbog principa lokalnosti, pošto se blok podataka preneo na nivo M1 , u narednomperiodu očekuje se više pristupa lokacijama u tom bloku -> brži pristup
- Prosečno vreme pristupa nekom podatku:

$$Ts = H * T1 + 1 - H * (T1 + T2)$$

  - Ts- prosečno vreme pristupa za sistem
  - T1 - vreme pristupa za nivo M1 (npr. keš memorija)
  - T2 - vreme pristupa za nivo M2 (npr. glavna memorija)
  - H – stopa pogodaka (eng. hit). Pogodak je slučaj kada se podatak nađe u bržojmemoriji M1
- Ako postoji jaka lokalnost mogude je ostvariti velike stope pogodaka i sa relativnomalom veličinom keš memorije
- Keš memorija od 1K do 128K reči je uopšteno odgovarajuda, dok se glavna memorijaizražava u gigabajtima
- Zaključak: relativno mali nivo M1 daje velike stope pogodaka zahvaljujudi lokalnosti

➔ Pristupanje memoriji sa vise nivoa

- Primer memorije sa 3 nivoa: jedan nivo keš memorije M1 , glavna memorija M2 isekundarna memorija M3
- Prosečno vreme pristupa nekom podatku:

$$Ts = h1 * t1 + 1 - h1 * h2 * t2 + 1 - h1 * 1 - h2 * h3 * t3$$

  - hi - stopa pogodaka na nivou Mi (uslovna verovatnoda da se podaci nalaze na nivouMi pod uslovom da se ne nalaze na nivou Mi-1)
  - ti - ukupno vreme potrebno za pristupanje podacima na nivou Mi . Jednako je zbirusvih vremena na putanji do pogotka na nivou Mi

II – 3 Logička i fizička keš memorija

- Logička keš memorija:
  - Poznata i kao virtuelna keš memorija
  - Podatke skladišti upotrebom virtuelnih adresa
  - Procesor pristupa logičkoj keš memoriji direktno, bez prolaska kroz MMU
- Fizička keš memorija:
  - Podatke skladišti upotrebom fizičkih adresa glavne memorije
- Prednosti logičke keš memorija:
  - brži pristup podacima jer nema prevođenja adresa •
- Nedostatak logičke keš memorije: svim aplikacijama se uglavnom dodeljuje isti prostorvirtuelnih adresa npr. virtuelna adresa 0
  - Potrebni dodatni bitovi da se označi na koji virtuelni prostor ukazuje adresa

Ili se keš memorija mora prazniti (eng. flushing cache) sa svakom promenomkonteksta aplikacija

IV – 11 Straničenje po zahtevu. Virtuelna memorija.

- Korišćenjem straničenja, efikasnost sistema za višeprogramski rad dolazi do

punogizražaja

- Straničenje po zahtevu (demand paging) označava da se određena stranica učitava umemoriju samo kad je potrebna tj. na zahtev
- Zamislimo slučaj velikog procesa koji se sastoji od dugačkog programa i više nizovapodataka. Tokom nekog kraćeg vremenskog perioda izvršavanje može da bude
  ograničeno na manji deo programa (npr. neki potprogram) i možda jedan niz podataka
- Učitavanje desetina stranica procesa bilo bi neefiksano trošenje resursa. Umesto togamožemo učitati tek nekoliko stranica procesa
- Zatim ako se program grana na instrukciju na stranici koja nije u glavnoj memoriji ili ukazuje na podatke koji nisu u memoriji, pokreće se prekid nazvan greška stranice kojimse OS-u saopštava da učita traženu stranicu
- Tako se u bilo kom trenutku u memoriji nalazi tek nekoliko stranica svakog od procesa izato je moguće držati više procesa u memoriji
- Štedi se i vreme jer se nekorišćene stranice ne premeštaju u memoriju niti iz memorije
- Kada se neka nova stranica učita u memoriju, mora se neka stara stranica izbaciti ->zamena stranica
- Algoritmi za zamenu starnica
  - LRU –algoritam najdavnije korišćene stranice (kao kod keš memorije)
  - U praksi se LRU teško realizuje za straničenje memorije, već se koriste malo drugačijialgoritmi
- Straničenjem po zahtevu moguće je da proces bude veći od čitave glavne memorije
  - -> programer ne brine da li mu je program suviše velik tj. da li ima dovoljnomemorije
  - Bez straničenja, programer bi morao da realizuje svoj veliki program iz delova koji bise jedan po jedan učitavali u memoriju
  - -> straničenjem po zahtevu taj posao je sad prepušten OS
  - -> straničenjem po zahtevu, programer “vidi“ ogromnu memoriju koja je jednakaprostoru koji joj je dodeljen na disku
- Procesi se izvršavaju samo u glavnoj memoriji -> realna memorija
- Programeri imaju utisak da je memorija mnogo veća, onoliko koliko je prostora za njuizdvojeno na disku -> virtuelna memorija

Virtuelna memorija omogućava efikasan višeprogramski rad i oslobađa korisnikeograničenja glavne memorije

V – 10 Mark compact algoritam – njegove faze i algoritmi koji ga predstavljaju:

- Algoritmi se sastoje od tri faze:
  - Označavanje (mark),
  - Kompaktovanje,
  - Ažuriranje referenci

a. Two-finger kompaktovanje – opis obe faze i objašnjenje
   - Osmišljen od strane Edvarda Sauncers-a 1974,
   - Koristi se za kompaktovanje objekata fiksne veličine, inače ne rezultuje punomdefragmentacijom
   - Ovaj algoritam koristi proizvoljni redosled objekata nakon kompaktovanja
   - I. Faza kompaktovanja:
     - 1. Sastoji se od kretanja dva pokazivača jednog (free) sa početka heap-a i drugog(scan) sa kraja heap-a,
     - 2. Prvi pokazivač se kreće dok ne nađe slobodan prostor, a drugi smanjuje dok se nenađe zauzet prostor,
     - 3. Objekat na koji pokazuje scan se kopira na lokaciju na koju pokazuje pokazivač free, a na staru lokaciju objekta se upisuje njegova nova adresa (pokazivač na novulokaciju),
     - 4. Kada se free i scan pokazivači poklope – proces kompaktovanja je završen
   - II. Faza ažuriranja referenci:
     - 1. Proći kroz sve reference na objekte smeštene na heap-u,
     - 2. Ukoliko referenca pokazuje iznad vrednosti free pokazivača, koju on ima nakonfaze kompaktovanja, onda:
     - 3. Referencu ažurirati novom vrednošću zabeleženom na staroj lokaciji objekta.
     - Nakon ažuriranja svih referenci, oslobađa se sva memorija iznad pokazivača free
b. Lisp 2 kompaktovanje – opis faza i objašnjenje
   - klizajući alogoritam kompaktovanja
   - Dodaje se polje u zaglavlju svakog objekta za označavanje prosleđujuće adrese -> to jeoverhead i mana ovog algoritma
   - Koristi se na objektima različite veličine
   - Sastoji se od tri prolaza kroz heap:
   - 1. 1. Računanje nove adrese gde će se svaki dostupni objekat naći nakon kompaktovanjai 1.2. čuvanja te adrese unutar svakog objekta u polju forwardingAddress
   - 2. Ažuriranje referenci korena i markiranih objekata na vrednosti forwardingAddress
   - 3. Pomeranje objekata na izračunate nove lokacije
   - Smer prolaska kroz heap (na gore) pri traženju novih lokacija je suprotan smerupremeštanja objekata (na dole)
   - Ovo garantuje da kada se objekat pomera (u 3. prolazu algoritma) ta lokacija je većprazna
   - dva pokazivača (free i live) kreću sa dna heap-a
   - aktivni objekti se premeštaju na slobodno mesto zadržavajući originalni redosled

VI – 4 Osnovne karakterisitke TCP protokola. TCP logički tok

- TCP je protokol pouzdanog prenosa koji se nalazi u transportnom sloju.
- Radi na vrhu IP koji je deo mrežnog sloja.

- Kako je IP nepouzdan protokol, TCP realizuje mehanizme kontrole toka i prenosa kojiobezbeđuju pouzdan prenos informacije.
- TCP potvrđuje svaki paket koji primi, tako da pošiljaoc zna da je paket stigao na svojeodredište.
- U slučaju da TCP sa predajne strane ne primi potvrdu, on ponovo šalje nepotvrđenipaket (eng. retransmission).
- Na taj način aplikacija može računati da će mreža obaviti pouzdanu isporuku informaciježeljenom odredištu.

➔ TCP logicki tok

- TCP je stream-oriented, jer sve podatke u prenosu tretira prosto kao niz okteta.
- Isporuka na odredištu jeste identična po broju i redosledu okteta, ali ne i po porukamana ulazu u socket

VII – 15 Tipičan tok CUDA programa

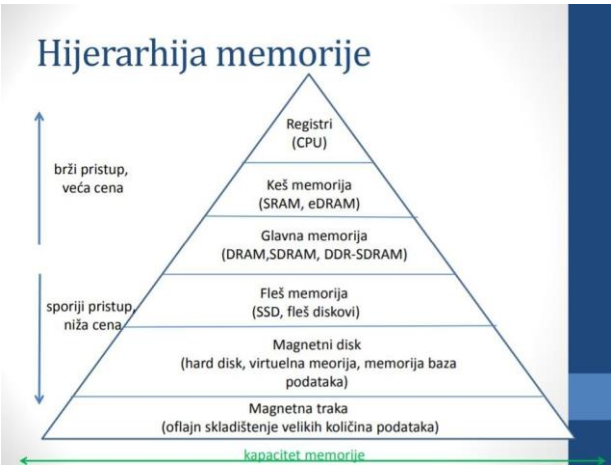
1. Load data into CPU memory
   fread/rand
2. Copy data from CPU to GPU memory
   cudaMemcpy(..., cudaMemcpyHostToDevice)
3. Call GPU kernel
   yourkernel<<>>(...)
4. Copy results from GPU to CPU memory.
   cudaMemcpy(...,
   cudaMemcpyDeviceToHost)
5. Use results on CPU

VII – 17 Kernel i hijerarhija niti

- Kernel se izvršava u paraleli preko skupa paralelnih niti
- Sve niti koje su generisane startovanjem kernela se kolektivno zovu rešetka
- Niti su organizovane u blokove, a blokovi su organizovani u rešetke
- Blok niti je skup konkurentno izvršavanih niti koje mogu da sarađuju između sebe putembarrier – sinhronizacije i deljene memorije
- Rešetka je niz blokova niti koji izvršavaju isti kernel
  - Čitaju ulaze i upisuju rezultate u globalnu memoriju
  - Sinhronizuju se između zavisnih kernel poziva

APRIL

I – 3 Hijerarhija memorije – objasnjenje piramide memorijske hijerarhije



➔ Kretanjem naniže kroz hijerarhiju memorije dešava se sledede:
- Smanjuje se cena po bitu
- Povedava se kapacitet
- Povedava se vreme pristupa
- Procesor ređe pristupa memoriji
  - Ovo se postiže iskorišdenjem principa lokalnosti

➔ Manje, skuplje i brže memorije se dopunjavaju vedim, jeftinijim i sporijim memorijama

➔ Moguce je organizovati podatke duž ove hijerarhije memorije tako da procenat pristupa zasvaki naredni nivo bude značajno manji od nivoa iznad

➔ Tipičan računarski sistem je opremljen hijerarhijom memorijskih podsistema, od kojih suneki unutrašnji (procesor im direktno pristupa), a neki su spoljašnji (procesor im pristupapreko U/I modula)

II – 2 Čitanje iz keš memorije – pogodak i promašaj u keš memoriji

- Procesor generiše adresu reči koja treba da se pročita
- Ako se ta reč nalazi u keš memoriji ->pogodak u keš memoriji -> reč se isporučuje do procesora
- Ako se desi promašaj u keš memoriji, dve stvari moraju da se dese:

• Pristupanje glavnoj memoriji za blok koji sadrži traženu reč (adresu)

• Dodeljivanje linije keš memorije za blok glavne memorije koji de se kopirati

• 1. Princip „prvo najvažnija reč“ : tražena reč se isporučuje procesoru

• 2. zatim se u keš memoriju učitava blok glavne memorije koji sadrži tu reč

- Keš memorija je povezana sa procesorom preko linija za podatke, adrese i kontrolu

IV – 10 Straničenje memorije i adresiranje – objašnjenje sadržaja logičke adrese i njeno prevođenje ufizičku adresu

- U slučaju straničenja memorije, logička adresa obuhvata dva podatka: broj stranice irelativnu adresu unutar te stranice
- Prevođenje logičke u fizičku adresu i dalje obavlja procesor
- Procesor mora da zna kako da pristupi tabeli stranica tekućeg procesa
- Procesor korišćenjem logičke adrese (broj stranice, relativna adresa unutar stranice) :
- 1. prvo pristupa tabeli stranica datog procesa da bi za dati broj stranice saznao brojokvira u koji je stranica smeštena
- 2. zatim pristupa podatku u glavnoj memoriji koristeći dati broj okvira memorije irelativnu adresu unutar stranice (okvira)

V – 9 Problem fragmentacije i kompaktovanje memorije

- Problem fragmentacije:
  - Nema dovoljno memorije pri alociranju većih objekata
  - Alokacija postaje teža i neefikasnija
  - Duža je faza brisanja
  - Gubi se na lokalnosti – objekti alocirani zajedno imaju težnju da im se pristupa zajedno. Mešanjem novih alociranih objekata sa „starim“ narušava se lokalnost
- Algoritmi kompaktovanja memorije rešavaju ove probleme premeštajući sve dostupneobjekte u jedan kompaktan deo heap-a
- Uopšteni zadatak kompaktovanja:
  - Premestiti dostupne objekte u jednu oblast heap-a
  - Izmeniti pokazivače tako da referenciraju objekte na novim lokcijama
- Redosled objekata pri kompaktovanju može biti :
  - Proizvoljan (Arbitrary) - jednostavno, ali narušava raspored objekata na heap-u iprostornu lokalnost
  - Linearizujući (Linearizing) - grupiše objekte koji međusobno imaju reference
  - Klizajući (Sliding) - čuva trenutni raspored objekata na heap-u (lokalnost) -> najčešćekorišćen u modernim kolektorima

VI – 5 TCP sesija – uspostava sesije i prekid sesije

- Definisana je sa dva para [IP adresa, port], za procese na oba kraja konekcije
- Sesija za svaki par računara u TCP vezi
- Uspostava prethodi razmeni paketa, 3 koraka

➔ Uspostava sesije

- Klijent inicira, Server prihvata konekciju
- Razmena inicijalnih (random) vrednosti brojača sekvence

➔ Prekid sesije

- Obe strane mogu prekinuti sesiju, ali to obično radi klijent
  - Active / pasive close
- Razmenjuju se FIN i ACK paketi

VII – 15 Tipičan tok CUDA programa

1. Load data into CPU memory
   fread/rand
2. Copy data from CPU to GPU memory
   cudaMemcpy(..., cudaMemcpyHostToDevice)
3. Call GPU kernel
   yourkernel<<>>(...)
4. Copy results from GPU to CPU memory.
   cudaMemcpy(...,
   cudaMemcpyDeviceToHost)
5. Use results on CPU

VII – 20 Sinhronizacija u CUDA jeziku: __syncthreads() i cudaDeviceSynchronize()

- 1.___syncthreads() sinhronizuje niti unutar bloka
  - Mora da bude izvršena od strane svih niti u bloku
  - To je if naredba: ili sve niti u bloku izvršavaju putanju koja uključuje syncthreads()ili ni jedna
- 2. cudaDeviceSynchronize() sinhronizuje sve niti u rešetki
- Za niti iz različitih rešetki, sistem omogučuje writes od kernela koji se dešavaju pre readsod sledeće “pokrenute” rešetke
- 3. Atomic operacije predupređuju konflikte pridružene nitima koje konkurentnopristupaju varijabli
  - Atomic operacije su primenjene nad varijablama i u globalnoj i u deljenoj memoriji
  - Npr. float atomicAdd(float* addr, float amount)
- Sve niti unutar vorpa izvršavaju se u lockstep-u. SM zapravo izvršava istu instrukciju uparaleli u isto vreme samo na različitim podacima. Niti su eksplicitno sinhronizovane
  - Lockstep sistemi su fault-tolerant sistemi koji izvršavaju isti set operacija u istovreme u paraleli
- __syncthreads() se koristi unutar kernela. Obezbeđuje barijeru za sinhronizaciju nitiunutar bloka
  - stopira nit dok sve ostale niti ne stignu do te barijere lokacije
- Koristi se da bi se izbegli RAW (read after write) / WAR / WAW hazardi kada se pristupadeljenoj ili globalnoj memoriji
- Kada se piše uslovni kod veoma je važno da se omogući da su sve niti sposobne dastignu do __syncthreads(); U suprotnom dolazi do potpunog zastoja (deadlock) programa.

- Dozvoljena je upotreba u uslovnim konstrukcijama (conditional constructs) samo ako jeuslov - kondicional uniforman kroz ceo blok
- U glavnom programu main() možemo da sinhronizujemo sve na host-u i device-u dojedne tačke koda:
  cudaDeviceSynchronize(); poziv sa hosta koji je blokiran sve dok se GPU kerneli nekompletiraju
- Ovo omogućava da se svi asinhroni (zovu se i ne-blokirajući) taskovi završe pre nego štose izvršenje koda nastavi
- cudaStreamSynchronize() čeka dok se svi kerneli u stream-u (niz operacija) ne završe

Nema sinhronizacije između blokova